COMPONENTE CURRICULAR:	Desenvolvimento de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

Etapa 1 – Compreensão Conceitual

a) A refatoração é uma técnica disciplinada de reestruturar o código para melhorar sua organização interna, legibilidade e manutenção, mas sem alterar o que ele faz na prática para o usuário. A relação com o *bad smell* (mau cheiro) é que o "mau cheiro" é o sintoma que indica que o código precisa de refatoração. Não é um erro de lógica, mas um código "feio" ou confuso que dificulta o trabalho futuro.

b)

1. **Long Method (Método Longo):** Quando um método faz coisas demais e fica gigante. Técnica: **Extract Method**.
2. **Duplicated Code (Código Duplicado):** Quando a mesma lógica aparece em vários lugares. Técnica: **Pull Up Method** (se estiverem em subclasses) ou **Extract Method**.

c) O objetivo da refatoração é puramente estrutural. Se você muda o comportamento, você não está mais refatorando, está alterando funcionalidades ou corrigindo bugs. O risco de não respeitar isso é introduzir novos erros (regressão) em partes do sistema que já estavam funcionando perfeitamente, gerando instabilidade.

Etapa 2 – Análise e Proposta de Refatoração

a)

1. **Long Method:** O método imprimirRelatorio está fazendo o trabalho de formatação do cabeçalho, a iteração da lista, a formatação de cada livro e o rodapé. Ele tem muitas responsabilidades.

2. **Data Clumps (ou falta de encapsulamento):** As informações do livro (título, autor, ano) estão espalhadas em três arrays diferentes. Eles sempre aparecem juntos, o que indica que deveriam estar unidos.

b)

- **Extract Method:** Eu aplicaria no imprimirRelatorio. Criaria um método imprimirDetalhesLivro(int i) para cuidar apenas da parte que imprime as informações de cada obra dentro do loop.
- **Extract Class:** Criaria uma classe Livro com os atributos titulo, autor e ano. Isso transformaria os três arrays confusos em uma única lista de objetos Livro.

c) O maior problema é o uso de **Arrays Paralelos**. Você depende de que o índice [0] de um array corresponda exatamente ao índice [0] dos outros. Se alguém ordenar o array de títulos e esquecer de ordenar o de autores, os dados ficam errados. Isso torna a manutenção perigosa e o reuso impossível, pois você não pode passar um "objeto livro" para outra parte do sistema.

Etapa 3 – Pesquisa e Expansão

a) Técnica escolhida: Replace Conditional with Polymorphism (Substituir Condicional por Polimorfismo).

b)

- **Bad smell:** Resolve o uso excessivo de switch ou longas cadeias de if-else que dependem do tipo de um objeto para decidir o que fazer.
- **Contexto:** Recomendada quando você tem uma lógica que varia de acordo com o tipo do objeto (ex: calcular bônus diferente para Gerente e Estagiário).
- **Exemplo:** * *Antes:* Um if que verifica if (tipo == "Moto") para calcular o pedágio.

- *Depois:* Uma classe abstrata Veiculo com o método calcularPedagio(), onde as classes Moto e Caminhao implementam sua própria versão do cálculo.

c) Essa técnica é a essência do polimorfismo. Em vez de perguntar ao objeto "quem é você?" para saber como agir (condicional), você simplesmente diz "faça sua parte" (método abstrato). O sistema se torna mais limpo porque, para adicionar um novo tipo, você só cria uma nova classe em vez de caçar todos os if/else do código.

Etapa 4 – Síntese e Reflexão Pessoal

a) Acredito que seja o medo de quebrar algo que já funciona e a falta de testes automatizados. Em códigos antigos, muitas vezes não há documentação, e uma pequena mudança pode gerar um "efeito dominó" em partes do sistema que o desenvolvedor nem sabia que estavam conectadas.

b) Não é recomendado quando o código está tão bagunçado que é mais barato e rápido reescrevê-lo do zero. Também não se deve refatorar próximo a prazos de entrega críticos, pois o risco de introduzir um erro inesperado é alto demais para o momento.

c) Imagine que o código é o seu quarto. Refatorar não é colocar móveis novos ou mudar de casa, mas sim organizar o que já está lá para que você encontre tudo mais rápido e tenha espaço para crescer. Sem refatoração, o código vira uma bagunça tão grande que, em pouco tempo, você vai gastar mais tempo tentando entender o que escreveu do que criando coisas novas.